

PROGRAMMAZIONE II

COMPILAZIONE, ESECUZIONE E RILASCIO IN JAVA

Michele Pasqua, PhD



UNIVERSITÀ
di **VERONA**
Dipartimento
di **INFORMATICA**

A.A. 2025/2026

IL LINGUAGGIO JAVA

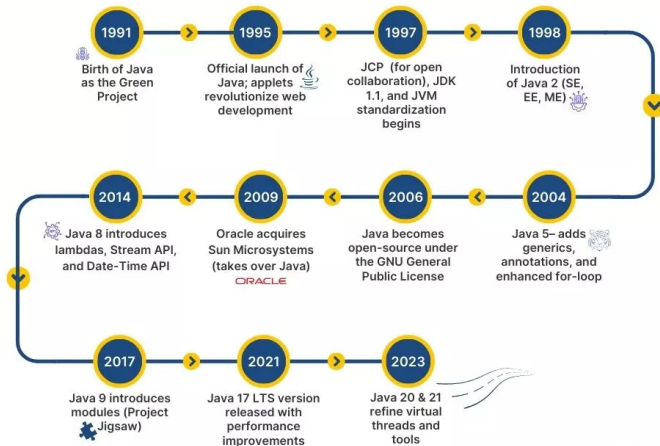


BREVE STORIA DI JAVA

Cronologia di Java



James Gosling



CARATTERISTICHE DI JAVA

Java è un linguaggio robusto, ossia meno incline agli errori

- ▶ Fortemente tipizzato // controlli a tempo di compilazione
- ▶ Nessuna manipolazione esplicita dei puntatori // meno errori di memoria
- ▶ Controlli a tempo di esecuzione // nessun traboccamento di array
- ▶ Garbage collection automatica // meno perdite di memoria (memory leak)
- ▶ Gli errori vengono gestiti dai programmatori tramite eccezioni

Java è un linguaggio dinamico

// non nel senso della teoria dei linguaggi

- ▶ Caricamento e collegamento a tempo di esecuzione
- ▶ Dimensione dinamica degli array

CARATTERISTICHE DI JAVA (CONT.)

Java è facile da imparare e usare

- ▶ Sintassi vicina a C/C++
- ▶ Linguaggio orientato agli oggetti quasi puro
- ▶ Facile compilazione e deployment

Java è portabile

- ▶ Indipendente dalla piattaforma
- ▶ Tradotto in un linguaggio intermedio e poi interpretato

Java non è lento come dicono (molte ottimizzazioni, es. JIT)

1B iterazioni di cicli
annidati

	C/clang -O3 (0.50s)
	Rust (0.50s)
	Java (0.54s)
	Kotlin (0.56s)
	Go (0.80s)
	Js/Bun (0.80s)
	Js/Node (1.03s)
	Js/Deno (1.06s)
	Dart (1.34s)
	PyPy (1.53s)
	PHP (9.93s)
	Ruby (28.80s)
	R (73.16s)
	Python (74.42s)

JAVA NON È JAVASCRIPT

«Java sta a JavaScript come l'auto (Car) sta al tappeto (Carpet).»

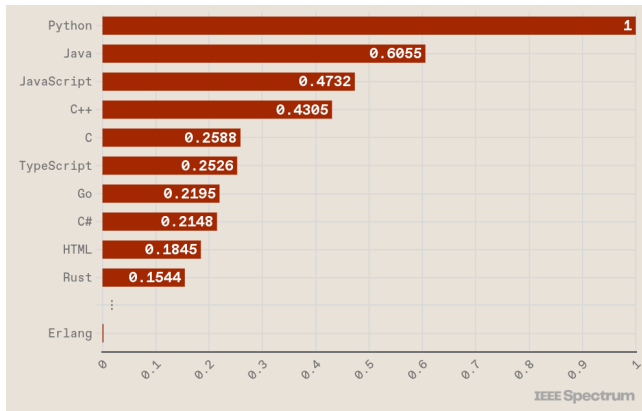
Christian Heilmann

Il nome JavaScript fa parte di una strategia di marketing di Netscape



POPOLARITÀ DI JAVA

I principali linguaggi di programmazione nel 2024



PORTABILITÀ: IL BYTECODE

Java è un linguaggio interpretato

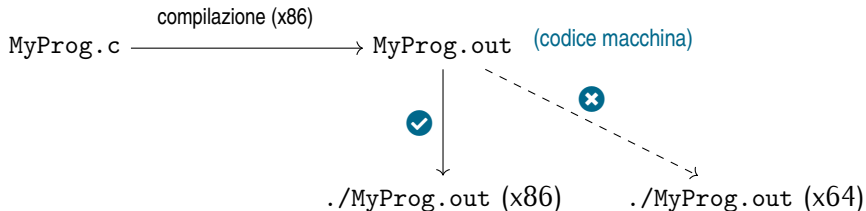
- ▶ Il compilatore non traduce direttamente in codice macchina
- ▶ Il codice sorgente viene tradotto in un linguaggio intermedio detto bytecode
- ▶ Il bytecode viene interpretato sulla macchina di destinazione tramite la JVM

Write once, run everywhere

- ▶ Invece di distribuire eseguibili (per ciascuna macchina di destinazione)
- ▶ Si distribuisce un interprete (per ciascuna macchina di destinazione)
- ▶ In questo modo, lo stesso bytecode può essere eseguito su tutte le macchine

COMPILATO VS INTERPRETATO

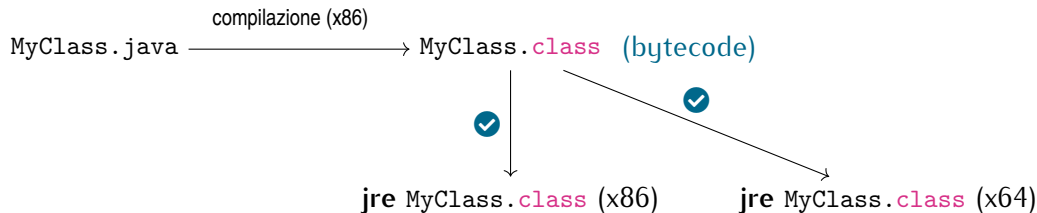
Deployment con un linguaggio compilato (come C/C++)



☹️ Il supporto cross-platform è a carico del programmatore

COMPILATO VS INTERPRETATO (CONT.)

Deployment con un linguaggio interpretato (come Java)



Il supporto cross-platform è a carico dello sviluppatore del linguaggio

ECOSISTEMA JAVA

... ciò che puoi usare per scrivere programmi Java

Java Development Kit – jdk

- ▶ Debugger Java (jdb)
- ▶ Compilatore Java (javac)
- ▶ Disassemblatore Java (javap)
- ▶ Documentazione Java (javadoc)

Java Runtime Environment – jre

- ▶ Librerie di classi (API standard)
- ▶ Compilatore Just-In-Time (JIT)
- ▶ Java Virtual Machine (JVM)
- ▶ Avviatore di applicazioni Java (java)

DEPLOYMENT PROGRAMMI JAVA



PRIMO PROGRAMMA JAVA

Un programma Java consiste in una classe, da salvare in un file con lo stesso nome

```
// Il mio primo programma Java -- Da salvare in un file di nome MyClass.java
public class MyClass {

    // punto di ingresso (entrypoint) del programma
    public static void main(String[] args) {
        System.out.println("Hello_World!");
    }

}
```

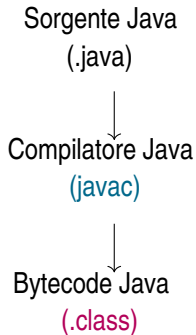
1
2
3
4
5
6
7
8
9

Per generare il bytecode: `javac MyClass.java`

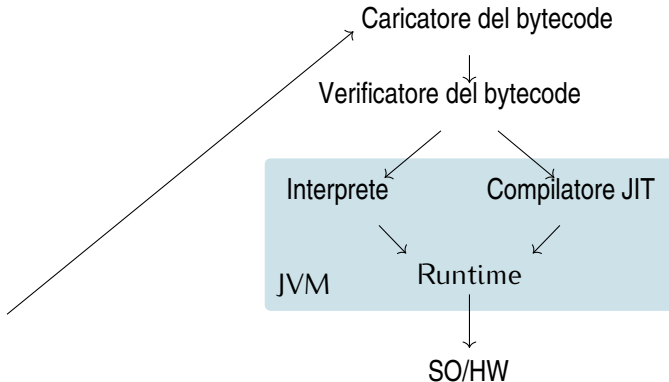
► Questo creerà un file denominato `MyClass.class`

COMPILAZIONE ED ESECUZIONE

Ambiente di compilazione (jdk)



Ambiente di runtime (jre)



CARICAMENTO DINAMICO DELLE CLASSI

Il caricamento delle classi nella JVM si basa sul classpath

- ▶ Elenco delle posizioni da cui è possibile caricare le classi

Quando è necessaria una classeClazz

- ▶ Cerca il file Clazz.class nella prima posizione del classpath
- ▶ Se non è presente, cerca nella posizione successiva del classpath

CARICAMENTO DINAMICO DELLE CLASSI (CONT.)

```
// Il mio primo programma Java -- Da salvare in un file di nome MyClass.java
public class MyClass {

    // punto di ingresso (entrypoint) del programma
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }

}
```

1
2
3
4
5
6
7
8
9

Per eseguire il bytecode: **java -cp . MyClass**

- ▶ Indica alla JVM di cercare le classi nella directory corrente (ossia .)
- ▶ System.out non è in ., quindi la JVM lo cercherà nel classpath di sistema

DEPLOYMENT

I programmi Java sono solitamente impacchettati e distribuiti in file jar

- ▶ Archivi compressi contenenti il bytecode
- ▶ Contengono meta-informazioni aggiuntive
- ▶ La JVM può caricare le classi da un file jar

Per creare un file jar: `jar cvf MyJar.jar MyClass.class`

- ▶ È possibile impacchettare più classi, es. *.class

Per eseguire una classe da un file jar: `java -cp MyJar.jar MyClass`

DEPLOYMENT (CONT.)

Se in un file jar è definita una classe principale, basta digitare: **java -jar MyClass.jar**

Per definire una classe principale, occorre aggiungere un file manifest personalizzato

jar cvfm MyClass.jar manifest.txt MyClass.class

dove manifest.txt contiene la riga **Main-Class: MyClass**

Il file manifest viene memorizzato nel file jar come: **META-INF/MANIFEST.MF**

AMBIENTE DI SVILUPPO JAVA



EDITOR DI TESTO

Prima di tutto, è necessario installare il jdk // di solito il jre è incluso nel jdk

apt install default-jdk oppure **apt install openjdk-17-jdk**

Poi, basta un semplice editor di testo per programmare // come gedit, vim o emacs

Flusso di lavoro

- ▶ Salvare un programma in uno o più file .java
- ▶ Compilare i file Java con javac per ottenere il bytecode (file .class)
- ▶ Facoltativamente impacchettare il bytecode in un archivio jar
- ▶ Eseguire la JVM sul bytecode con java

AMBIENTE DI SVILUPPO INTEGRATO

Un IDE è una toolchain a supporto dei programmatori

IDE disponibili per Java

- ▶ IntelliJ IDEA <https://www.jetbrains.com/idea/download>
- ▶ Eclipse <https://www.eclipse.org/downloads>
- ▶ NetBeans <https://netbeans.apache.org/front/main/download>
- ▶ Visual Studio Code <https://code.visualstudio.com/docs/languages/java>

AMBIENTE DI SVILUPPO INTEGRATO (CONT.)

Vantaggi di un IDE

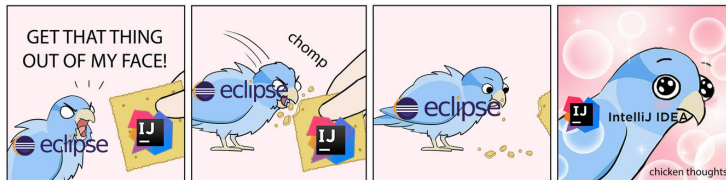
- ▶ Fornisce l'evidenziazione della sintassi (syntax highlighting)
- ▶ Fornisce linting del codice in tempo reale // controllo dei tipi, code smell, ecc.
- ▶ Aggiunge il supporto alla formattazione (pretty printing)
- ▶ Semplifica l'esecuzione del codice e i test // debugger integrato
- ▶ Offre organizzazione e navigazione grafica del progetto
- ▶ Integra strumenti di controllo versione // git
- ▶ Integra strumenti di compilazione/build // per Java: maven, gradle o ant
- ▶ Fornisce generazione automatica di codice, da non usare durante l'apprendimento

QUALE IDE?



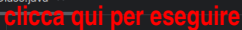
QUALE IDE? (CONT.)

Puoi scegliere l'IDE che preferisci, in laboratorio useremo IntelliJ IDEA



È un software proprietario, tuttavia

- ▶ La Community Edition è gratuita per tutti
- ▶ La Ultimate Edition è gratuita per gli studenti universitari





DOMANDE E RISPOSTE

